23BEC0503

Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

SCHOOL OF ELECTRONICS ENGINEERING

Fall Semester 2025-2026

BECE406E – FPGA Based System Design

TASK 5 – FPGA implementation of DSP Application

SANJAY ELAVARASAN KARTHIKEYAN

23BEC0503

Aim_1:

Design a 4 tap - serial FIR filter and 4 tap - pipelined FIR filter with filter coefficients of h0=4, h1=3, h2=2, h1=1. Verify the functionality using modelsim and justify which design is preferred. Implement only 4 tap - serial FIR filter in FPGA board. Attach screenshots of code and output, RTL view and FPGA board implementation.

Software Details:

- Functional Simulation: Modelsim
- Synthesis & Implementation: Intel Quartus Prime 23.1

Hardware Details:

- Family: Cyclone IV E
- Package: FBGA
- Pin count: 780
- Core speed grade – 7
- Device - EP4CE115F29C7

23BEC0503

RTL Code for SERIAL 4-TAP FIR FILTER :

C:/Users/SANJAY ELAVARASAN/Desktop/FPGA/FPGA_DA_5/fir_serial_23bec0503.v (/tb_fir_filter/uut) - Default

```
//SANJAY ELAVARASAN KARTHIKEYAN - 23BEC0503
module fir_filter(
input clk,
input reset,
input [3:0] SW,
output reg [7:0] LEDR
);
parameter N = 4;
reg signed [3:0] h[0:N-1];
reg signed [3:0] SW_reg [0:N-1];
reg [2:0] count;
always @(posedge clk or posedge reset) begin
if (reset) begin
// Reset filter output and shift register
LEDR <= 8'd0;
h[0] <= 4'sh4;
h[1] <= 4'sh3;
h[2] <= 4'sh2;
h[3] <= 4'sh1;
SW_reg[0] <= 4'h0;
SW_reg[1] <= 4'h0;
SW_reg[2] <= 4'h0;
SW_reg[3] <= 4'h0;
end
else begin
SW_reg[3] <= SW_reg[2];
SW_reg[2] <= SW_reg[1];
SW_reg[1] <= SW_reg[0];
SW_reg[0] <= SW; // Store the new input at the first position
if(count < 3'b100)
LEDR <= LEDR + (h[count] * SW_reg[count]);
end
end
always @(posedge clk or posedge reset)
if(reset)
count <= 3'b0;
else
count <= count + 3'b1;
endmodule
```

23BEC0503

Testbench for SERIAL 4-TAP FIR FILTER :

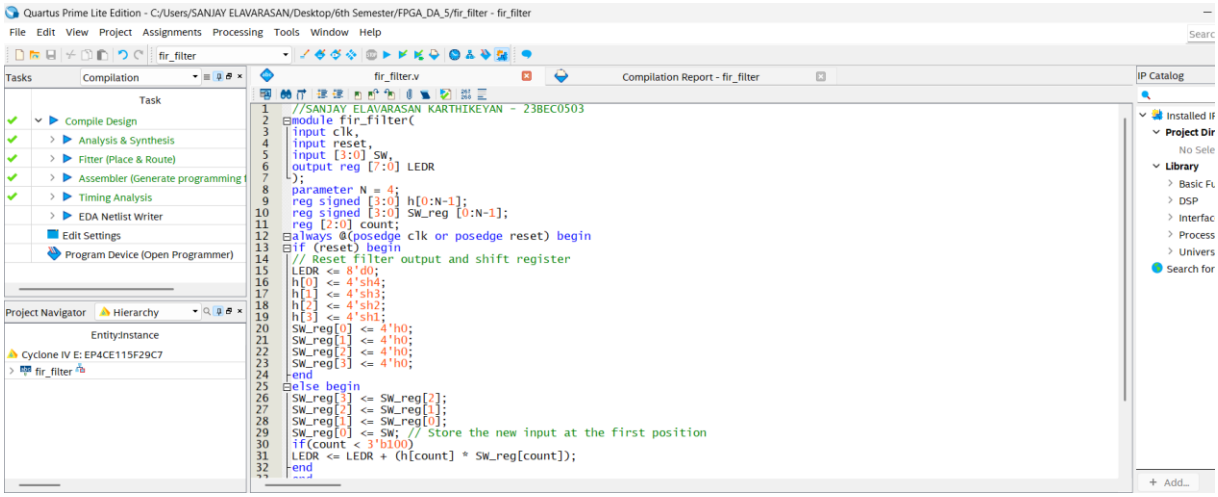
C:/Users/SANJAY ELAVARASAN/Desktop/FPGA/FPGA_DA_5/serial_tb_23bec0503.v (/tb_fir_filter) - Default

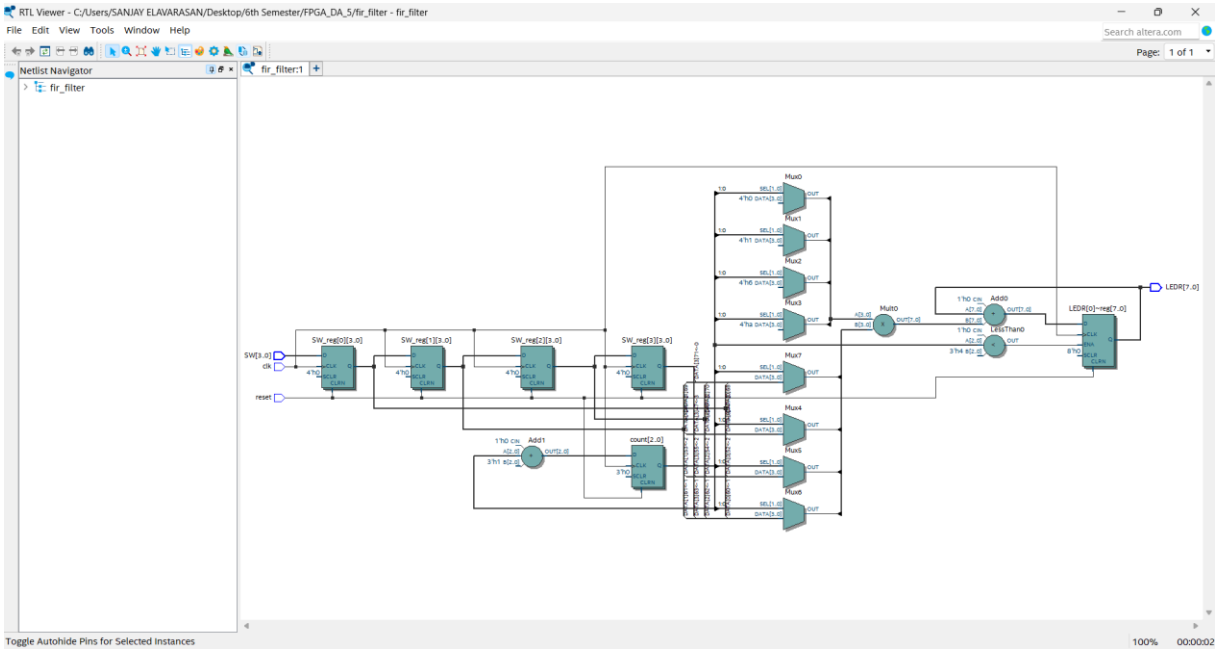
```
//SANJAY ELAVARASAN KARTHIKEYAN - 23BEC0503
`timescale 1ns/1ps
module tb_fir_filter;
reg clk;reg reset;
reg [3:0] SW;
wire [7:0] LEDR;

// DUT
fir_filter uut (
    .clk(clk),
    .reset(reset),
    .SW(SW),
    .LEDR(LEDR)
);

// Clock
always #5 clk = ~clk;

initial begin
    clk = 0;
    reset = 1;
    SW = 4'd4;
    $monitor("Time=%0t | SW=%d | LEDR=%d", $time, SW, LEDR);
    #20 reset = 0;
    #200;
    $display("Done");
    $finish;
end
endmodule
```

Functional Verification - Output Waveform :

23BEC0503

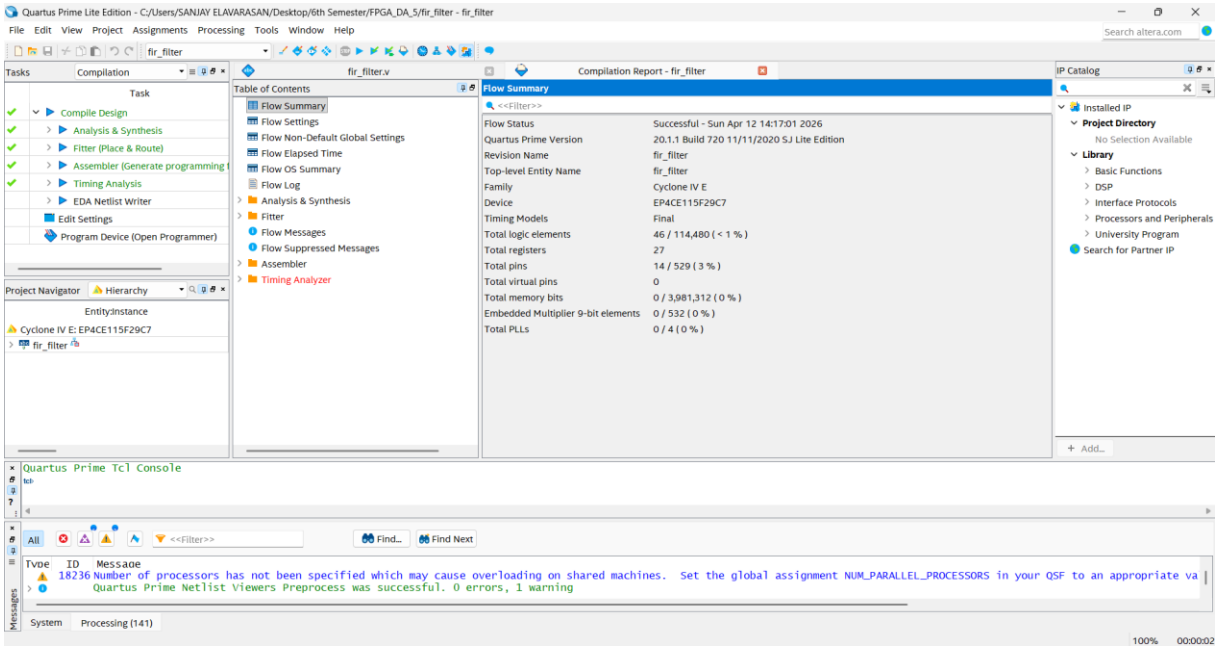
RTL VIEW :





23BEC0503

COMPILATION REPORT :



FPGA IMPLEMENTATION

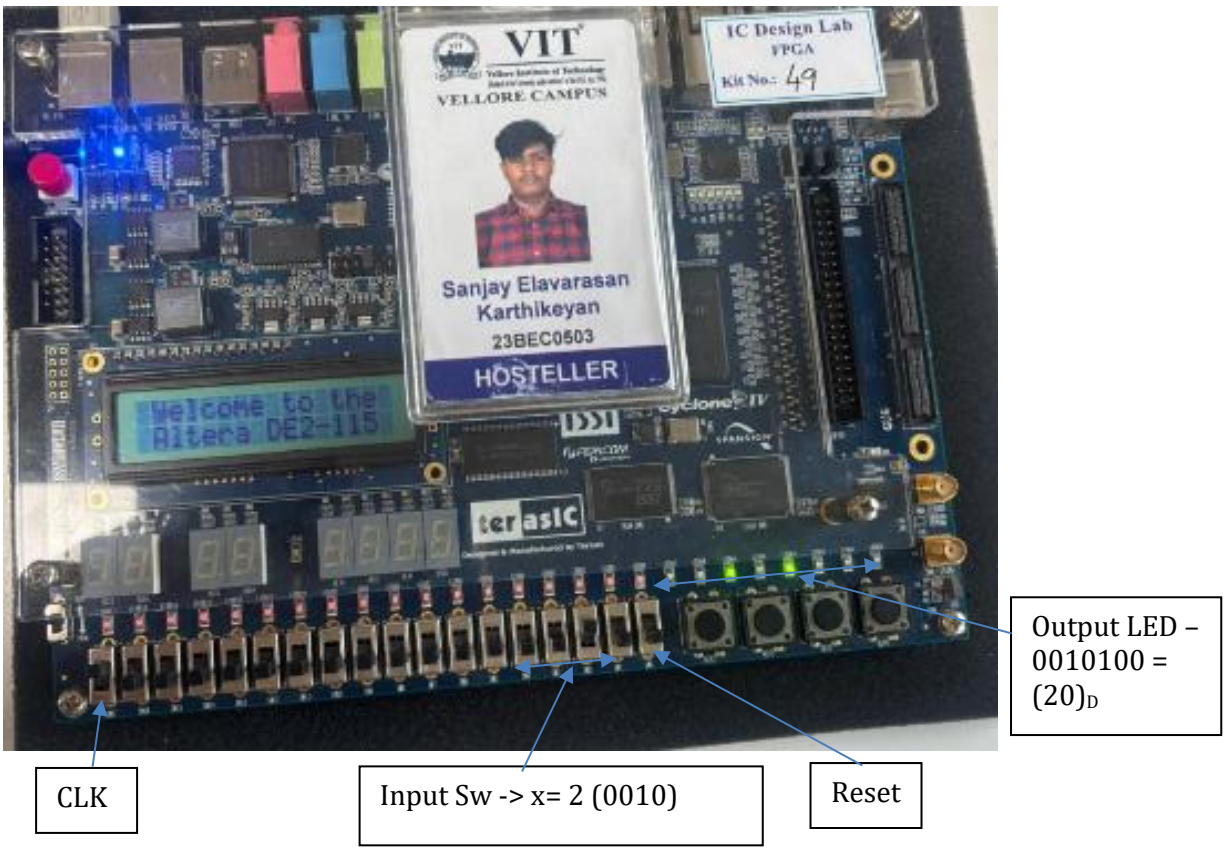
Output LED – 0101000 = $(40)_D$

CLK

Input Sw –> x= 4 (0100)

Reset

23BEC0503



RTL Code for 4-TAP PIPELINED FIR FILTER :

C:/Users/SANJAY ELAVARASAN/Desktop/FPGA/FPGA_DA_5/fir_pipe_23bec0503.v (/tb_fir_filter_pipe/uut)

File Edit View Tools Bookmarks Window Help

```
//SANJAY ELAVARASAN KARTHIKEYAN - 23BEC0503
module fir_filter_pipe(
    input  clk,
    input  reset,
    input  [3:0] SW,
    output reg [7:0] LEDR
);
parameter N = 4;
// Fixed coefficients: h = {4, 3, 2, 1}
wire signed [3:0] h [0:N-1];
assign h[0] = 4'sh4;
assign h[1] = 4'sh3;
assign h[2] = 4'sh2;
assign h[3] = 4'sh1;
// Shift register to hold last N inputs
reg signed [3:0] SW_reg [0:N-1];
// — PIPELINE STAGE 1 — Multiplication results
reg signed [7:0] mul [0:N-1];
// — PIPELINE STAGE 2 — Adder tree result (registered output)
// ————————————————————————————————
// Stage 0 : Shift register (runs every clock)
// ————————————————————————————————
integer i;
always @(posedge clk or posedge reset) begin
    if (reset) begin
        SW_reg[0] <= 4'sh0;
        SW_reg[1] <= 4'sh0;
        SW_reg[2] <= 4'sh0;
        SW_reg[3] <= 4'sh0;
    end else begin
        SW_reg[0] <= $signed(SW);
        SW_reg[1] <= SW_reg[0];
        SW_reg[2] <= SW_reg[1];
        SW_reg[3] <= SW_reg[2];
    end
end
// ————————————————————————————————
```

23BEC0503

```
// ------------------------------------------------
// Stage 1 : Parallel multiply all taps at once
// ------------------------------------------------
always @(posedge clk or posedge reset) begin
    if (reset) begin
        mul[0] <= 8'sh00;
        mul[1] <= 8'sh00;
        mul[2] <= 8'sh00;
        mul[3] <= 8'sh00;
    end else begin
        mul[0] <= h[0] * SW_reg[0];    // 4 * x[n]
        mul[1] <= h[1] * SW_reg[1];    // 3 * x[n-1]
        mul[2] <= h[2] * SW_reg[2];    // 2 * x[n-2]
        mul[3] <= h[3] * SW_reg[3];    // 1 * x[n-3]
    end
end

// ------------------------------------------------
// Stage 2 : Add all products → final output
// ------------------------------------------------
always @(posedge clk or posedge reset) begin
    if (reset)
        LEDR <= 8'd0;
    else
        LEDR <= mul[0] + mul[1] + mul[2] + mul[3];
end

endmodule
```

Testbench for PIPELINED 4-TAP FIR FILTER :

C:/Users/SANJAY ELAVARASAN/Desktop/FPGA/FPGA_DA_5/pipe_tb_23bec0503.v (/tb_fir_filter_pipe)

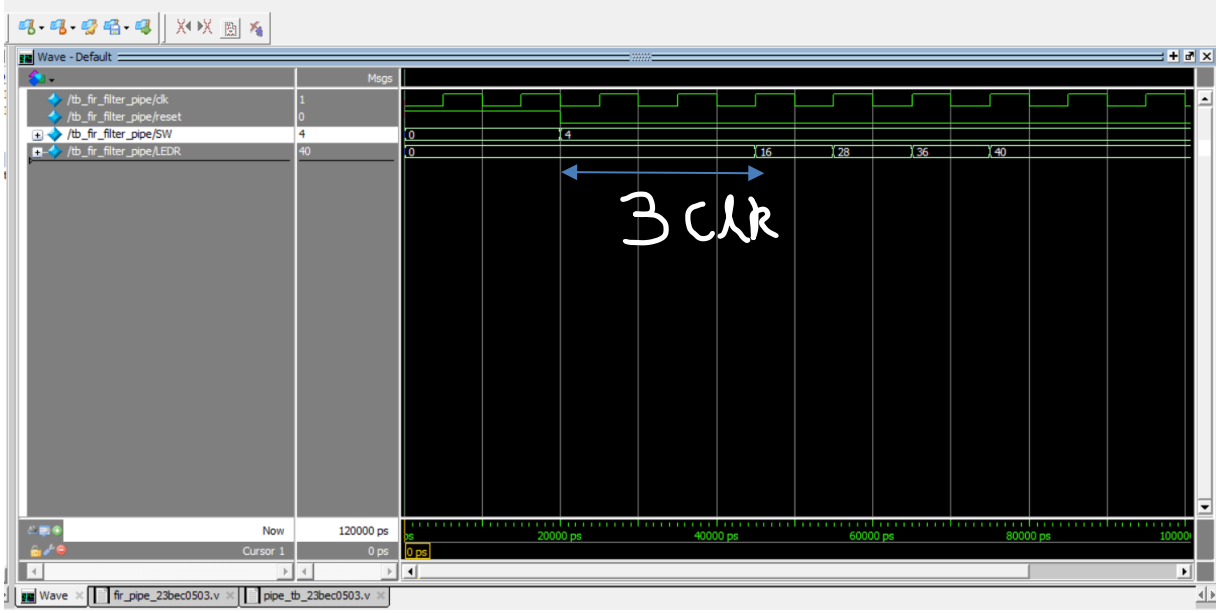
File Edit View Tools Bookmarks Window Help

C:/Users/SANJAY ELAVARASAN/Desktop/FPGA/FPGA_DA_5/pipe_tb_23bec0503.v (/tb_fir_filter_pipe) - Default*

```
//SANJAY ELAVARASAN KARTHIKEYAN - 23BEC0503
`timescale 1ns/1ps
module tb_fir_filter_pipe;

    reg        clk;
    reg        reset;
    reg  [3:0] SW;
    wire [7:0] LEDR;

    // -- DUT ------------------------------
    fir_filter_pipe uut (
        .clk   (clk),
        .reset (reset),
        .SW    (SW),
        .LEDR  (LEDR)
    );

    // -- Clock: 10 ns period ---------------
    initial clk = 0;
    always  #5 clk = ~clk;

    // -- Monitor every clock edge ----------
    always @(posedge clk)
        $display("Time=%0t | SW=%0d | LEDR=%0d", $time, SW, LEDR);

    // -- Stimulus --------------------------
    initial begin
        // ---- Reset phase ----
        reset = 1;
        SW    = 4'd0;
        #20;
        reset = 0;

        // ---- Apply constant input SW = 4 ----
        SW = 4'd4;
        #100;  // 10 extra cycles to clearly see steady state

        // -- Self-checking assertion -----
        if (LEDR == 8'd40)
            $display("PASS: LEDR = %0d (expected 40)", LEDR);
        else
            $display("FAIL: LEDR = %0d (expected 40)", LEDR);
    end
endmodule
```

23BEC0503

Functional Verification - Output Waveform



Serial Design — Limitations

The serial FIR filter performs one operation per clock cycle, so it takes multiple cycles (4 cycles for a 4-tap filter) to produce one output, resulting in low throughput. It also requires extra control logic like counters and accumulator reset, making the design more complex and prone to errors.

Pipelined Design — Advantages

The pipelined FIR filter pre-computes all multiplications in parallel and then adds them, allowing it to produce one output every clock cycle after an initial delay. It has simpler control logic, better speed, and is more suitable for high-performance applications.

Conclusion

Preferred Design:
The pipelined design is preferred because it offers much higher throughput, simpler design, and better performance, making it ideal for real-time signal processing.

Aim 2:

Design edge detection algorithm using Sobel operator and verify the functionality using modelsim with test cases showing no edge, strong edge (Horizontal/vertical), moderate edge. Attach Screenshots of code and outputs.

Software Details:

- Functional Simulation: Modelsim
- Synthesis & Implementation: Intel Quartus Prime 23.1

Hardware Details:

- Family: Cyclone IV E
- Package: FBGA
- Pin count: 780
- Core speed grade – 7
- Device - EP4CE115F29C7

23BEC0503

RTL Code :

File Edit View Tools Bookmarks Window Help

C:/Users/SANJAY ELAVARASAN/Desktop/6th Semester/FPGA_DA_5/Sobel edge/sobel_23bec0503.v (/th_sobel_edge_detection/uut) - Default

```
// SANJAY ELAVARASAN KARTHIKEYAN - 23BEC0503
module sobel_edge_detection #(parameter WIDTH = 8) (
    input wire clk,
    input wire rst,
    input wire [WIDTH-1:0] pixel_in,
    input wire [WIDTH-1:0] pixel_in_top,
    input wire [WIDTH-1:0] pixel_in_bottom,
    input wire [WIDTH-1:0] pixel_in_left,
    input wire [WIDTH-1:0] pixel_in_right,
    input wire [WIDTH-1:0] pixel_in_tl,
    input wire [WIDTH-1:0] pixel_in_tr,
    input wire [WIDTH-1:0] pixel_in_bl,
    input wire [WIDTH-1:0] pixel_in_br,
    output reg [WIDTH-1:0] edge_out,
    output reg ready
);

// Sobel kernels
reg signed [WIDTH-1:0] sobel_x [0:2][0:2];
reg signed [WIDTH-1:0] sobel_y [0:2][0:2];

// Gradients
reg signed [15:0] gradient_x;
reg signed [15:0] gradient_y;
reg signed [15:0] gradient_magnitude;

// Initialise kernels
initial begin
    sobel_x[0][0] = -1; sobel_x[0][1] = 0; sobel_x[0][2] = 1;
    sobel_x[1][0] = -2; sobel_x[1][1] = 0; sobel_x[1][2] = 2;
    sobel_x[2][0] = -1; sobel_x[2][1] = 0; sobel_x[2][2] = 1;

    sobel_y[0][0] = -1; sobel_y[0][1] = -2; sobel_y[0][2] = -1;
    sobel_y[1][0] = 0;  sobel_y[1][1] = 0;  sobel_y[1][2] = 0;
    sobel_y[2][0] = 1;  sobel_y[2][1] = 2;  sobel_y[2][2] = 1;
end

always @(posedge clk or posedge rst) begin
    if (rst) begin
        gradient_x <= 0;
        gradient_y <= 0;
        gradient_magnitude <= 0;
        edge_out <= 0;
        ready <= 0;
    end
    else begin
        // X gradient
        gradient_x <= (sobel_x[0][0] * pixel_in_tl) + (sobel_x[0][1] * pixel_in_top) + (sobel_x[0][2] * pixel_in_tr) +
                      (sobel_x[1][0] * pixel_in_left) + (sobel_x[1][1] * pixel_in) + (sobel_x[1][2] * pixel_in_right) +
                      (sobel_x[2][0] * pixel_in_bl) + (sobel_x[2][1] * pixel_in_bottom) + (sobel_x[2][2] * pixel_in_br);

        // Y gradient
        gradient_y <= (sobel_y[0][0] * pixel_in_tl) + (sobel_y[0][1] * pixel_in_top) + (sobel_y[0][2] * pixel_in_tr) +
                      (sobel_y[1][0] * pixel_in_left) + (sobel_y[1][1] * pixel_in) + (sobel_y[1][2] * pixel_in_right) +
                      (sobel_y[2][0] * pixel_in_bl) + (sobel_y[2][1] * pixel_in_bottom) + (sobel_y[2][2] * pixel_in_br);

        // Magnitude (approx)
        gradient_magnitude <= (gradient_x * gradient_x) + (gradient_y * gradient_y);

        // Output scaling
        edge_out <= (gradient_magnitude[15:8] > 8'd255) ? 8'd255 : gradient_magnitude[15:8];

        ready <= 1;
    end
end

endmodule
```

23BEC0503

TESTBENCH FOR FUNCTIONAL VERIFICATION:

File Edit View Tools Bookmarks Window Help

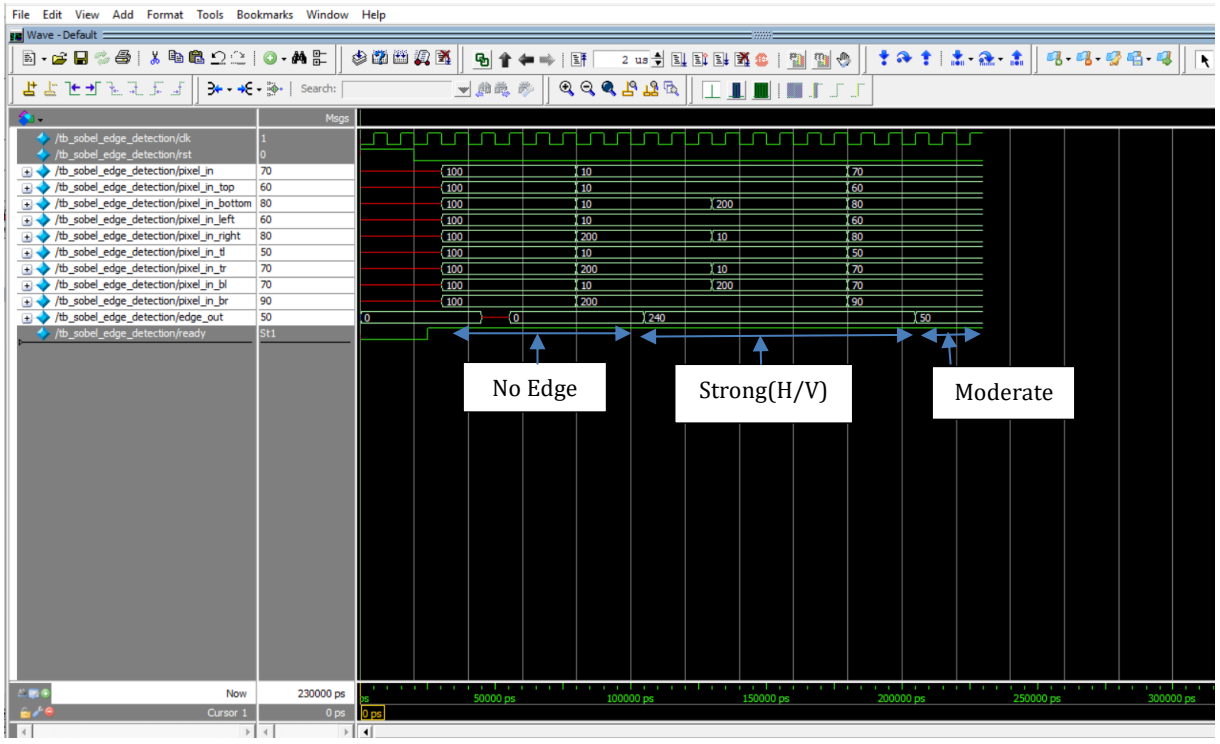
C:/Users/SANJAY ELAVARASAN/Desktop/6th Semester/FPGA_DA_5/Sobel edge/tb_sobel_23bec0503.v (/tb_sobel_edge_detection) - Default

```
`timescale 1ns / 1ps
// SANJAY ELAVARASAN KARTHIKEYAN - 23BEC0503
module tb_sobel_edge_detection;

parameter WIDTH = 8;

reg clk;
reg rst;
reg [WIDTH-1:0] pixel_in, pixel_in_top, pixel_in_bottom;
reg [WIDTH-1:0] pixel_in_left, pixel_in_right;
reg [WIDTH-1:0] pixel_in_tl, pixel_in_tr, pixel_in_bl, pixel_in_br;

wire [WIDTH-1:0] edge_out;
wire ready;

sobel_edge_detection #(WIDTH) uut (
    .clk(clk), .rst(rst),
    .pixel_in(pixel_in),
    .pixel_in_top(pixel_in_top),
    .pixel_in_bottom(pixel_in_bottom),
    .pixel_in_left(pixel_in_left),
    .pixel_in_right(pixel_in_right),
    .pixel_in_tl(pixel_in_tl),
    .pixel_in_tr(pixel_in_tr),
    .pixel_in_bl(pixel_in_bl),
    .pixel_in_br(pixel_in_br),
    .edge_out(edge_out),
    .ready(ready)
);

// Clock
always #5 clk = ~clk;

initial begin
    clk = 0;
    rst = 1;

    // Monitor output
    $monitor("Time=%0t | Edge_out=%d | Ready=%b", $time, edge_out, ready);

    #20 rst = 0;
    #10;

    // Test Case 1: No Edge
```

```
    // Test Case 1: No Edge
    $display("Test Case 1: No Edge");
    pixel_in = 100; pixel_in_top = 100; pixel_in_bottom = 100;
    pixel_in_left = 100; pixel_in_right = 100;
    pixel_in_tl = 100; pixel_in_tr = 100; pixel_in_bl = 100; pixel_in_br = 100;
    #50;

    // Test Case 2: Vertical Edge (Strong)
    $display("Test Case 2: Strong Vertical Edge");
    pixel_in_tl = 10; pixel_in_top = 10; pixel_in_tr = 200;
    pixel_in_left = 10; pixel_in = 10; pixel_in_right = 200;
    pixel_in_bl = 10; pixel_in_bottom = 10; pixel_in_br = 200;
    #50;

    // Test Case 3: Horizontal Edge (Strong)
    $display("Test Case 3: Strong Horizontal Edge");
    pixel_in_tl = 10; pixel_in_top = 10; pixel_in_tr = 10;
    pixel_in_left = 10; pixel_in = 10; pixel_in_right = 10;
    pixel_in_bl = 200; pixel_in_bottom = 200; pixel_in_br = 200;
    #50;

    // Test Case 4: Moderate Edge
    $display("Test Case 4: Moderate Edge");
    pixel_in_tl = 50; pixel_in_top = 60; pixel_in_tr = 70;
    pixel_in_left = 60; pixel_in = 70; pixel_in_right = 80;
    pixel_in_bl = 70; pixel_in_bottom = 80; pixel_in_br = 90;
    #50;

    $display("Simulation Complete.");
    $finish;
end

endmodule
```

23BEC0503

Functional Verification - Output Waveform:




```
VSIM 15> run
# Time=0 | Edge_out=  0 | Ready=0
# Time=25000 | Edge_out=  0 | Ready=1
# Test Case 1: No Edge
# Time=45000 | Edge_out=  x | Ready=1
# Time=55000 | Edge_out=  0 | Ready=1
# Test Case 2: Strong Vertical Edge
# Time=105000 | Edge_out=240 | Ready=1
# Test Case 3: Strong Horizontal Edge
# Test Case 4: Moderate Edge
# Time=205000 | Edge_out= 50 | Ready=1
# Simulation Complete.
# ** Note: $finish    : C:/Users/SANJAY ELAVARASAN/Desktop/6th Semester/FPGA_DA_5/Sobel edge/tb_sobel_23bec0503.v(73)
#    Time: 230 ns  Iteration: 0  Instance: /tb_sobel_edge_detection
# 1
# Break in Module tb_sobel_edge_detection at C:/Users/SANJAY ELAVARASAN/Desktop/6th Semester/FPGA_DA_5/Sobel edge/tb_sobel_23bec0503.v

VSIM 16>
```

INFERENCE :

FIR Filter Design

The serial FIR filter is simple and uses fewer hardware resources but produces one output after multiple clock cycles, making it slower. The pipelined FIR filter, on the other hand, gives one output every clock cycle after a small delay, making it much faster and suitable for high-speed applications. Although it uses more hardware, it is the preferred design due to higher throughput and simpler control logic.

Sobel Edge Detection

The Sobel edge detection algorithm successfully identifies edges based on intensity changes in the input. It correctly distinguishes between no edge, moderate edge, and strong edges (horizontal and vertical). The simulation results confirm that the design works as expected and is effective for basic image edge detection.